

UGANDA CHRISTIAN UNIVERSITY

A Centre of Excellence in the Heart of Africa

Department of Computing & Technology

Investigate, Recommend or Escalate: A Rule-Constrained Reinforcement Learning Agent for IT Incident Triage under Ambiguous Evidence

By

OPOL CHARIS

Reg No: *S24B23/003*, Student No: *B29654*

E-mail: *charisopol@gmail.com*, Phone No: *0765821211*

A report submitted to the Faculty of Engineering Design & Technology for the study leading to the Degree in partial fulfilment of the requirements for the award of **Bachelor of Science in Computer Science** of Uganda Christian University

SPECIALIZATION TRACK: *Artificial Intelligence and Robotics*

IAN RAYMOND OSOLO

[Enter Version Date]

Contents

1 Introduction **1**
- 1.1 Background to the Study . . . 1
- 1.2 Problem Statement . . . 3
- 1.3 Research Objectives . . . 4
  - 1.3.1 General Objective . . . 4
  - 1.3.2 Specific Objectives . . . 4
- 1.4 Research Questions . . . 4
- 1.5 Research Hypotheses . . . 5
- 1.6 Scope of the Study . . . 5
- 1.7 Significance of the Study . . . 6
- 1.8 Justification . . . 7
- 1.9 Conceptual Framework . . . 7
- 1.10 Operational Definition of Key Terms . . . 8
- 1.11 Chapter Summary . . . 9

2 Literature Review **10**
- 2.1 Review Approach . . . 10
- 2.2 Representing Ambiguous Incident Triage in a Controlled Environment . . . 11
- 2.3 Reinforcement Learning for Investigating, Recommending and Escalating . . . 12
- 2.4 Rule-Based Safety Mechanisms and Operational Constraints . . . 13
- 2.5 Evaluation Measures and Baseline Policies . . . 15
- 2.6 Robustness under Ambiguity and Missing Evidence . . . 16
- 2.7 Synthesis of the Reviewed Literature . . . 17
- 2.8 Research Gap . . . 19
- 2.9 Chapter Summary . . . 19

3 Methodology **21**
- 3.1 Research Design . . . 21
- 3.2 Research Pipeline . . . 22
- 3.3 Scenario Modelling with the Directorate . . . 23
- 3.4 The Simulated Triage Environment . . . 24
  - 3.4.1 Incident Model . . . 24
  - 3.4.2 Observations and State . . . 24
  - 3.4.3 Actions . . . 25
  - 3.4.4 Reward . . . 25
  - 3.4.5 Ambiguity Controls . . . 26
  - 3.4.6 Reference Labels . . . 26
- 3.5 The Reinforcement Learning Agent . . . 26
- 3.6 The Rule-Based Safety Mechanism . . . 27

3.7 Baseline Decision Policies 28
3.8 Evaluation Metrics 29
3.8.1 Decision Quality 29
3.8.2 Investigation Efficiency 29
3.8.3 Escalation Behaviour 29
3.8.4 Constraint Compliance 30
3.8.5 Robustness 30
3.9 Experimental Design 30
3.10 Tools and Implementation 31
3.11 Validity and Limitations 31
3.12 Ethical Considerations 32
3.13 Chapter Summary 32

REFERENCES **37**

List of Figures

1.1 Conceptual framework of the rule-constrained reinforcement learning agent for ambiguity-aware IT incident triage. 8

3.1 The research pipeline of the study. 22

List of Tables

2.1 Comparison of the studies closest to this research. ✓ = addressed; ∘ = partly addressed; – = not addressed or not reported. Studies marked † were assessed from their abstracts. 18

3.1 Alignment of the specific objectives with the method and the evaluation. 22
3.2 Baseline decision policies and the approaches they represent. 28

Chapter 1

Introduction

This chapter introduces a study of how an automated agent should decide what to do when the evidence about an IT incident is incomplete or conflicting. It presents the background to IT incident triage, states the research problem, and sets out the objectives, research questions and hypotheses of the study. It then defines the scope, significance and justification of the research, describes the conceptual framework, and defines the key terms used in the chapters that follow.

1.1 Background to the Study

Organisations depend on IT services that are expected to remain available, and incident management is the process that restores those services when they fail. An incident is an unplanned interruption or outage of a service [1]. Chen et al. describe incident management as the process of detecting a service problem, creating an incident, determining its cause, restoring the service, and capturing knowledge to prevent a recurrence [1]. Failures that are handled slowly reduce system availability and can lead to financial loss and user dissatisfaction [1, 2]. The speed and quality of the decisions made early in this process therefore affect how long a service stays degraded.

Incident triage is the stage at which a reported incident is assessed and directed towards resolution. Chen et al. define triage as engaging the team responsible for investigating the problem [1]. Remil et al. use the term more broadly to cover the prioritisation, assignment and classification of incidents, together with the identification of duplicates [2]. Both descriptions present triage as a set of decisions made before the root cause is known. This study adopts the same view and treats ambiguity-aware IT incident triage as the task of deciding what should happen next to an incident whose cause is still uncertain.

Researchers and practitioners have moved triage from manual and rule-based processing towards learned models. Traditional IT management relied on expert systems and rule-based engines, which Remil et al. report as limited in adaptiveness, efficiency and scalability [2]. Artificial Intelligence for IT Operations (AIOps) addresses these limits with machine learning models that classify, prioritise and assign incidents from reports, logs and metrics [1, 2]. Large language models have extended this work to root cause analysis and the generation of mitigation suggestions from incident reports [3]. These approaches differ in what they hand to the person responsible. AI-assisted diagnosis proposes a cause, decision support recommends an action for a person to approve, automated execution carries the action out, and escalation passes the decision to a person. The literature is uneven across the four: Zhang et al. observe

that automation efforts mainly stop at incident triage or solution recommendation, and that work on automatic execution is limited and its practical effectiveness remains to be verified [3]. The move from recommending to acting is therefore the least settled part of AI-assisted incident handling.

Triage decisions are difficult because the evidence available at the time of the decision is often incomplete or conflicting. In their two-year study of incident management at Microsoft, Chen et al. found that symptoms are sometimes insufficient to identify the responsible team, since distinct problems can produce similar symptoms, and that incidents are consequently reassigned several times [1]. The same study attributes delays to incomplete knowledge of service dependencies and imprecise assessment of resource health [1]. Remil et al. likewise note that data in real IT environments is commonly noisy and incomplete, and they list robustness to such data as a requirement for AIOps solutions [2]. Many triage models nevertheless treat the task as a single prediction over the incident report, evaluated with classification or ranking metrics [2]. A model of this kind returns an answer for every report and has no means to state that more evidence is needed before it can answer. Ambiguity is therefore a property of the triage task that a single-step prediction does not represent.

Reinforcement learning offers a way to treat triage as a sequence of decisions and not as one prediction. In reinforcement learning, an agent learns a policy by acting in an environment and receiving rewards that reflect the consequences of its actions [4]. When the agent cannot observe the true state of the environment, it can choose actions whose purpose is to obtain information, and it must weigh the cost of that information against its value [5]. Janisch et al. applied this idea to classification with costly features, where an agent learns which features to acquire before it commits to a decision [6]. Reinforcement learning and related sequential methods have also been applied to incident handling. In IT operations, Wang et al. used reinforcement learning to reduce the number of questions put to engineers during root cause analysis [7]. In cybersecurity, Ren et al. trained an agent to select response strategies for security incidents [8], Dunsin et al. modelled post-incident malware investigation as a Markov decision process [9], Kim and Dán developed policies for choosing which alerts to investigate [10], and Hammar and Li planned responses when the model of the incident is itself uncertain [11]. These studies show that incident handling can be framed as a sequential decision problem and that the gathering of evidence can be made part of it. Their agents choose among responses or investigative steps, however, and the studies leave open how such an agent should behave when it is not permitted, or not well placed, to act.

An agent that recommends actions on production systems must respect operational constraints, and a reward signal alone does not ensure this. Reinforcement learning algorithms find policies that maximise reward but do not guarantee safe behaviour during learning or execution [12], and penalising undesirable actions in the reward does not guarantee that the agent never selects them [13]. Safe reinforcement learning addresses this by requiring policies to respect safety constraints as well as maximise return [14]. One group of methods constrains the expected cumulative cost of a policy [15, 16]. Many practical constraints, however, are instantaneous and must hold at every step [17]. Shielding is designed for this case: a separate component monitors each action the agent proposes and overrides it only when the action would violate a given specification [12]. The guarantee a shield provides is limited to the specification and the environment model from which it is built [13]. For incident handling, Remil et al. state the corresponding requirement directly: automatic healing actions need to be executed safely [2]. Two recent studies act on this requirement in security settings. Alauthman et al. propose a reinforcement learning agent with an explicit safety shield for incident response in industrial control systems [18], and Hammar et al. check the actions proposed by a language model against system constraints before they are used [19]. A rule of this kind in IT support might,

for example, forbid restarting a production database without approval.

Human experts remain necessary when ambiguity cannot be resolved or when a decision falls outside what the agent is permitted to do. Selective prediction gives a model the option to abstain when it is not sufficiently confident [20], and learning to defer extends this by training the model to decide when a human decision-maker should take over [21]. This research shows that deferral carries its own difficulties. Learning when to defer normally requires data about the expert's decisions [22], and the way a deferral is communicated affects the accuracy of the person who receives it [23]. Joshi et al. extended deferral to sequential settings, in which the choice to defer affects later states [24]. Guidance on AI risk management also recognises that an AI system may decide autonomously, defer to a human expert, or act as an additional opinion, and it requires that these roles be clearly defined [25]. Recent work outside IT operations supports making escalation an explicit and governed decision. Pacella et al. found that a language-model agent without a guardrail failed to escalate cases that policy required a human to approve [26], and Bachar et al. treated escalation in content moderation as a cost-aware decision based on uncertainty [27]. A recent preprint adds that human attention is finite, so an agent that escalates too often may weaken the oversight it relies on [28]. Escalation is consequently a decision with costs in both directions, and not a safe default.

These bodies of work have developed largely apart, and where they meet they do so in pairs. AIOps research has concentrated on predicting the category, owner or cause of an incident. Studies of information gathering let an agent choose between looking further and deciding [6,10], while studies of deferral let it choose between deciding and handing over [21, 24]. Shielding has been demonstrated mainly in control and robotics tasks [12, 15, 17], and the shielded agent proposed for incident response selects responses and does not choose whether to investigate or escalate [18]. The studies reviewed for this research do not examine an agent for IT incident triage that chooses among gathering further diagnostic information, recommending a permitted action, and escalating to a human expert, with its proposals checked by an independent set of operational rules. Nor do they evaluate such an agent as the ambiguity of its evidence is varied. This study addresses that opportunity. It proposes to design a rule-constrained reinforcement learning agent for ambiguity-aware IT incident triage and to evaluate it in a controlled simulation. Part of the study will be carried out with the Directorate of University ICT Services at Uganda Christian University, whose incident-handling practice is expected to inform the scenarios and operational constraints that the simulation represents.

1.2 Problem Statement

IT support teams must decide how to handle an incident before they know its cause. The evidence available at that point is often incomplete, and different observations can point towards different explanations [1,2]. A member of IT staff facing such an incident can investigate further, apply a likely remedy, or pass the incident to someone better placed to resolve it. Each choice has a cost. Further investigation takes time, a wrong remedy can prolong or worsen the incident, and unnecessary escalation consumes the attention of senior staff.

Existing automated approaches address parts of this decision but not the choice among its options. Triage models commonly produce one prediction per incident report [1, 2], which does not allow the model to ask for more evidence. Sequential methods have been applied to incident response and investigation, mostly in cybersecurity [8–11], and some of them decide what evidence to gather. A few studies of security alert triage also let the agent escalate or defer an alert to an analyst [29, 30], but their agents close or escalate alerts and do not recommend a remedy, and a systematic review of 34 studies found escalation to a person in only seven of

them [31]. Learning to defer provides a principled basis for handing decisions to humans [21,24], but it treats the choice as one between deciding and deferring, without the option of gathering information first. Shielding and constrained reinforcement learning provide ways to restrict an agent's actions [12, 15], and a shielded agent has been proposed for incident response in industrial control systems [18]. Only three of the 34 studies in the systematic review report what happens when such a mechanism blocks a decision [31], and the reviewed literature does not report how the mechanism behaves when the agent can also investigate or escalate.

The gap this study investigates is therefore specific, and Chapter Two sets it out in five parts. The reviewed literature offers no controlled environment for IT service incidents in which ambiguity can be varied. It does not show whether one policy can learn to choose among investigating, recommending and escalating, or how an independent rule-based mechanism interacts with that choice. It has not measured decision quality, investigation cost, escalation behaviour and constraint compliance together for one agent, nor has it shown how an agent's mix of decisions changes as evidence becomes scarcer or more contradictory. The study does not claim that constraining a learned incident-handling agent is new, or that no related work exists beyond the literature reviewed. This study proposes to build a simulated triage environment, informed in part by work with the Directorate of University ICT Services at Uganda Christian University, in which the question can be tested, and to compare a rule-constrained reinforcement learning agent with baseline decision policies. Whether the agent improves on those baselines is a question for the experiments and not an assumption of the study.

1.3 Research Objectives

1.3.1 General Objective

To design and evaluate a rule-constrained reinforcement learning agent for ambiguity-aware IT incident triage that learns when to gather additional diagnostic information, recommend permitted actions, or escalate incidents to a human expert.

1.3.2 Specific Objectives

1. To identify and model IT incident triage scenarios involving incomplete, uncertain, or conflicting diagnostic evidence.

2. To design a reinforcement learning environment in which an agent can request diagnostic information, recommend permitted actions, or escalate incidents.

3. To implement an independent rule-based safety mechanism that checks proposed actions against predefined operational constraints.

4. To evaluate the proposed approach against selected baseline decision policies using measures of decision quality, investigation efficiency, escalation behaviour, and constraint compliance.

5. To assess the robustness of the agent under different levels of ambiguity and missing diagnostic information.

1.4 Research Questions

This study will answer the following research questions:

1. How can ambiguous IT incident triage scenarios involving incomplete or conflicting diagnostic evidence be represented in a controlled environment?

2. How can reinforcement learning support decisions about gathering diagnostic information, recommending permitted actions, and escalating incidents to a human expert?

3. How effectively can a rule-based safety mechanism prevent actions that violate predefined operational constraints?

4. How does the proposed approach compare with selected baseline decision policies in decision quality, investigation efficiency, escalation behaviour, and constraint compliance?

5. How robust is the proposed approach when incident evidence is incomplete, conflicting, or increasingly ambiguous?

1.5 Research Hypotheses

This study will test the following hypotheses:

H1: The rule-constrained reinforcement learning agent will achieve higher decision quality than the selected baseline decision policies in ambiguous incident scenarios.

H2: With the independent rule-based safety mechanism, the agent will achieve a higher rate of rule-compliant correct recommendations than the same agent operating without the mechanism.

H3: The agent will request more diagnostic information and escalate more incidents as the level of ambiguity in the incident evidence increases.

1.6 Scope of the Study

This study concerns decision-making in AI-assisted IT incident triage. Its subject is the choice an agent makes among gathering diagnostic information, recommending an action that operational constraints permit, and escalating an incident to a human expert when the evidence is ambiguous. It does not address incident detection, root cause analysis as a separate task, or the long-term prevention of incidents.

The technical work is limited to four components: a simulated triage environment, a reinforcement learning policy, an independent rule-based safety mechanism, and a set of baseline decision policies. The simulated incidents will be drawn mainly from application errors of the kind that appear in the logs of the university's systems, together with the infrastructure causes, such as database and authentication failures, that surface in those logs. The categories are provisionally application and interface errors, authentication and access failures, database failures, and failures of infrastructure and external services, and they will be confirmed with the Directorate. The scenario model comprises about 15 causes, 20 diagnostic checks and 10 operational rules, and it will be constructed for the study.

Part of the study will be carried out with the Directorate of University ICT Services at Uganda Christian University, which has agreed to support it. The Directorate provides the organisational setting from which the study expects to draw its understanding of triage practice, including the kinds of incidents that are reported and the operational rules that govern how IT staff respond. An initial consultation, an interview with a senior developer of the Directorate

in October 2026, described how incidents reach developers through user tickets, vetting and assignment, and how application errors are observed through system logs and monitoring; it informed the choice of incident categories above. The form of further involvement, including any consultation with IT staff or use of anonymised historical incident records, will be agreed with the Directorate. It will require the Directorate's permission, ethical clearance where applicable, and appropriate data protection procedures. The study will not reproduce confidential incident information, credentials or personal data.

The evaluation is limited to controlled experiments in simulation. These experiments will measure decision quality, investigation cost, escalation behaviour, constraint compliance, and robustness to ambiguity and missing evidence. The human expert is represented in the simulation as the recipient of escalated incidents, and the experiments do not place the agent's decisions before working IT staff.

The study excludes deployment in a production environment, including the live systems of the Directorate, unrestricted automated remediation, and any claim about operational effectiveness in a real organisation. The agent recommends actions and does not execute them on live systems. The results will therefore describe the behaviour of the agent under the assumptions of the simulation.

1.7 Significance of the Study

The study may be useful to IT support teams by examining how an automated assistant could investigate before it recommends and escalate when the evidence does not support a recommendation. Evidence on when such an agent asks for more information, and at what cost, could inform how teams divide work between automated tools and IT staff. The findings may be of particular interest to the Directorate of University ICT Services at Uganda Christian University, whose practice informs the study. The initial consultation indicated that incidents there are routed to developers by assignment and that error logs are already passed to AI coding assistants in some cases, while the decision whether an incident should be handled automatically or brought to a person rests with staff. The study examines that decision, and the consultation also identified the security of incident data, and control over who can see it, as a condition for using such an agent.

For organisations, the study examines whether explicit operational rules can be enforced on a learning agent in incident workflows. Organisations that are cautious about automation in IT operations need evidence about how constrained agents behave before they consider adopting them, and a simulation study is a low-risk way to begin producing that evidence.

For researchers, the study offers an evaluation of reinforcement learning for ambiguity-aware IT incident triage, an area in which the reviewed literature concentrates on prediction and not on sequential decisions. The simulated environment and the evaluation measures may also serve as a starting point for later studies, and the systematic review conducted for the study [31] maps the evidence on how reinforcement learning agents for incident handling gather information, escalate and are constrained.

For developers of AI systems, the study may provide insight into the design and evaluation of agents that separate three kinds of decision: gathering information, recommending an action, and passing the decision to a person. These are potential contributions, and their extent depends on the results of the experiments.

1.8 Justification

This research is motivated by a decision that IT support teams face routinely and that existing automated approaches represent only in part. The literature shows that incident evidence is often insufficient to identify the cause or the responsible team [1], and that automation beyond recommendation remains limited and unverified [3]. A study that examines what an agent should do under such evidence addresses a documented difficulty and not a hypothetical one.

The combination of reinforcement learning and explicit operational constraints is worth evaluating because each compensates for a weakness of the other. Reinforcement learning can learn when further investigation is worth its cost, which fixed rules handle poorly, but it gives no assurance that the learned policy avoids prohibited actions [12]. Explicit rules give that assurance for the actions they cover, but they cannot decide which permitted action is best under uncertain evidence. Keeping the rules in a mechanism that is independent of the learned policy also makes the constraints inspectable, which matters where operational rules must be reviewed and changed by people.

The ability to recognise uncertainty and escalate appropriately matters because both kinds of error are costly. An agent that never escalates will sometimes recommend actions that its evidence does not support, and an agent that always escalates removes the benefit of automation and burdens the staff who receive its cases [28]. Measuring escalation behaviour alongside decision quality allows the study to examine this trade-off directly.

Finally, the study is feasible within a final-year project. A simulated environment allows controlled variation of ambiguity and missing evidence, repeated experiments, and comparison with baselines, without the risk of acting on a production system. Working with the Directorate of University ICT Services at Uganda Christian University grounds the simulated scenarios and constraints in the practice of an operating IT support unit, while the simulation keeps the experiments away from its live systems.

1.9 Conceptual Framework

The conceptual framework describes how the components of the proposed system relate to one another. Figure 1.1 presents these relationships as a decision loop between the agent and the simulated environment.

Each simulated incident has an underlying cause that the agent cannot observe directly. The agent receives incident observations instead: the initial symptoms and the results of any diagnostic checks it has requested. From these it maintains an estimate of the incident state, which summarises the evidence gathered so far. The reinforcement learning policy uses this estimate to propose one of three kinds of decision: request further diagnostic information, recommend an action, or escalate the incident to a human expert.

The independent rule-based safety mechanism checks every proposed decision against the predefined operational constraints before it takes effect. A proposal that satisfies the constraints passes to the environment unchanged, and a proposal that violates a constraint is blocked. The agent is told that its proposal was blocked and proposes again. If the second proposal also violates a constraint, the incident is escalated to the human expert. The environment then returns the simulated outcome. A diagnostic request returns new evidence at a cost, while a recommendation or an escalation ends the episode with an outcome that depends on the underlying cause. The reward derived from this outcome is the feedback from which the policy

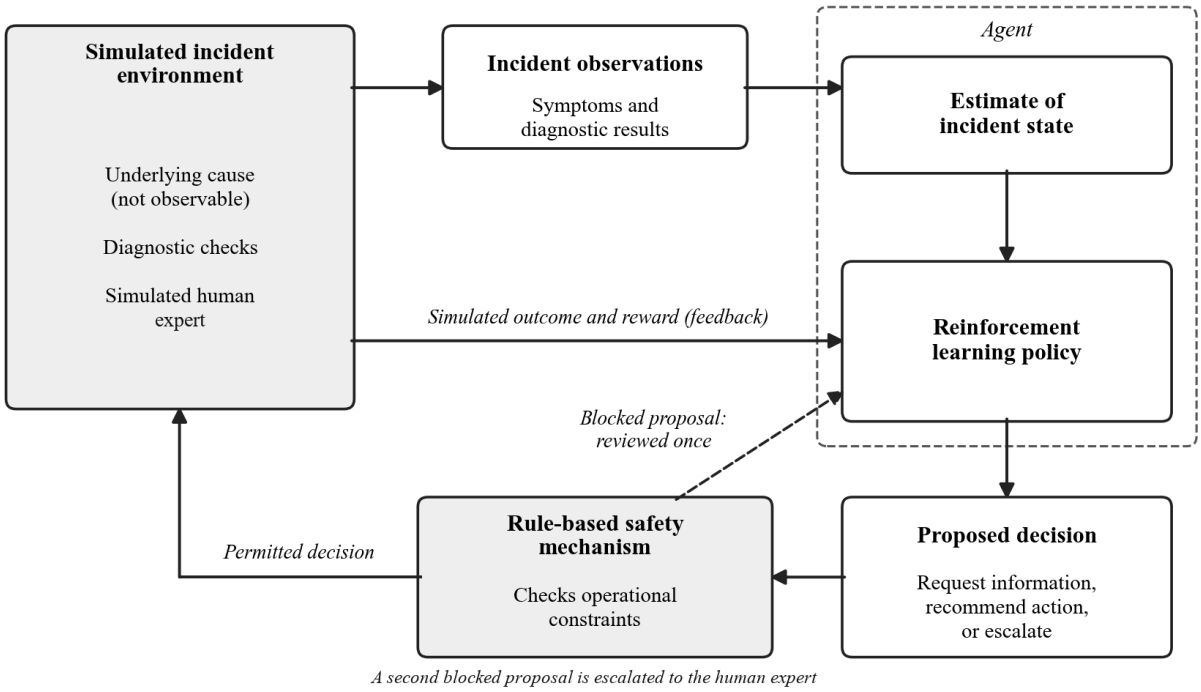


Figure 1.1: Conceptual framework of the rule-constrained reinforcement learning agent for ambiguity-aware IT incident triage.

learns.

The framework separates learning from rule enforcement. The policy learns from feedback which permitted decision suits the evidence, and the safety mechanism enforces the rules regardless of what the policy has learned. The mechanism enforces only the constraints that have been specified, within the simulated environment, and the framework does not imply a guarantee of safety in real operations.

1.10 Operational Definition of Key Terms

IT incident triage: The early stage of incident handling in which a reported incident is assessed and a decision is made about what should happen to it next. In this study, triage is the decision among gathering more diagnostic information, recommending a permitted action, and escalating to a human expert.

Ambiguity: A condition in which the available evidence about an incident is consistent with more than one plausible cause, because the evidence is incomplete, uncertain, or conflicting.

Incomplete diagnostic evidence: Evidence from which one or more observations relevant to identifying the cause of an incident are missing at the time a decision is required.

Reinforcement learning: A machine learning approach in which an agent learns a decision policy by interacting with an environment and receiving rewards for the outcomes of its actions [4]. In this study, the environment is a simulation of IT incident triage.

Diagnostic information gathering: An action by which the agent requests an additional observation about an incident, such as the result of a diagnostic check, before committing to a recommendation or an escalation. Each request carries a cost in the simulation.

Operational constraint: A predefined rule that specifies an action the agent is not permitted to recommend, either in general or under stated conditions.

Safety mechanism: A rule-based component, independent of the learned policy, that checks each proposed action against the operational constraints and blocks actions that violate them. A blocked proposal is returned to the agent once, and a second blocked proposal results in escalation. It corresponds to the notion of a shield in safe reinforcement learning [12]. In this study, "safety" refers to compliance with the specified constraints and not to safety in a broader sense.

Human escalation: The decision to pass an incident to a human expert instead of recommending an action. In this study, the human expert is simulated.

Baseline decision policy: A simpler decision procedure against which the proposed agent is compared, such as a fixed rule-based policy or a reinforcement learning agent without the safety mechanism.

1.11 Chapter Summary

This chapter has introduced the study. It showed that IT incident triage often proceeds on incomplete or conflicting evidence, that existing automated approaches mostly treat triage as a single prediction, and that research on reinforcement learning, operational constraints and human deferral has developed largely apart. It stated the problem as the lack of evidence on how a rule-constrained reinforcement learning agent chooses among investigating, recommending and escalating under ambiguity, and it set out the objectives, research questions, hypotheses, scope, significance, justification, conceptual framework and key terms of the study, and it identified the Directorate of University ICT Services at Uganda Christian University as the setting for part of the work. Chapter Two reviews the related literature in greater depth and establishes the research gap more fully.

Chapter 2

Literature Review

This chapter reviews the literature that bears on the five research questions of the study. It first describes how the literature was gathered and organised. It then synthesises the literature under five themes, one for each research question, and in each theme it separates what the reviewed studies establish from what they leave open. The chapter closes by comparing the studies closest to this one and by stating the research gap that the study addresses.

2.1 Review Approach

The review draws on three bodies of literature that Chapter One identified as relevant: AI-assisted IT incident management, safe and constrained reinforcement learning, and the deferral of decisions from automated systems to humans. A core set of 34 publications was collected across these areas. It comprises surveys of AIOps, empirical studies of incident management, foundational and recent work on shielding and constrained reinforcement learning, studies of selective prediction and learning to defer, and recent work on the runtime governance of language-model agents. Four further sources supply the standard formulations of reinforcement learning, partial observability, costly information acquisition and sequential deferral on which the discussion relies [4–6, 24].

A systematic literature review, conducted for this study and reported separately [31], supplemented this core set. It followed the PRISMA 2020 guideline. Searches of Scopus, the ACM Digital Library and IEEE Xplore on 23 October 2026 returned 4,399 records. After the removal of duplicates and of records that failed the document-type, year and search-term filters, 131 records were screened on title and abstract, 47 reports were sought, 40 were assessed in full, and 34 studies that apply reinforcement learning or sequential decision-making to IT or security incident handling were included. The studies of that review that bear most directly on this one are discussed in the themes below from their full texts, and its counts are used where they show how common a practice is. Three studies in the core set, by Kim and Dán, Hammar et al. on language-model planning, and D'Antonio and Xie, were not part of the systematic review and are discussed on the basis of their abstracts [10, 19, 32].

The synthesis is thematic. Each study was read for what it contributes to one or more of the research questions, and the findings were grouped into five themes that correspond to those questions: the representation of ambiguous incidents, sequential decisions about investigating, recommending and escalating, rule-based safety mechanisms, evaluation measures and baselines, and robustness under ambiguity. The systematic review has its own limits: it searched three databases, its IEEE Xplore search was restricted to open-access records, citation search-

ing was not carried out, and seven reports could not be retrieved [31]. The conclusions of this chapter about gaps are therefore stated for the literature reviewed, and not as claims that the questions have never been examined.

2.2 Representing Ambiguous Incident Triage in a Controlled Environment

The first research question asks how ambiguous triage scenarios can be represented in a controlled environment. The literature answers part of this question by describing the evidence on which triage decisions rest. Remil et al. catalogue the data sources of incident management as incident reports, alerting signals, logs, metrics, traces and topology [2], and Zhang et al. add that language models have brought human-written material such as incident reports and documentation into use [3]. Incident reports are the primary source for the assignment of incidents [2]. An environment for triage must therefore represent two kinds of evidence: the initial report, and the further observations that a member of IT staff could obtain from logs, metrics and other diagnostic sources.

The literature also explains why this evidence is ambiguous. Chen et al. found that distinct problems can produce similar symptoms, so that the symptoms alone do not identify the responsible team, and that incomplete knowledge of dependencies and imprecise assessment of resource health delay diagnosis [1]. Remil et al. describe real IT data as commonly noisy and incomplete [2]. Wu and Gong describe a related condition for agents that act in organisations: the facts needed to judge whether an action complies with policy may be absent from what the agent can see [33]. Ambiguity in these accounts has three sources, which are missing observations, unreliable observations, and observations that are consistent with more than one cause. A controlled environment that claims to represent ambiguity should be able to produce each of them.

Realistic incident data is difficult to obtain, and this limits how the reviewed studies represent the task. Chen et al. report their results as relative values because of company policy [1]. Remil et al. observe that publishing datasets raises confidentiality concerns, and that most methods for assignment and deduplication have been developed on software bug reports and not on operational incidents [2]. Studies of sequential decision-making have responded by building or borrowing controlled environments. Dunsin et al. built a laboratory of infected virtual machines [9], Hammar and Li evaluated on the CAGE-2 benchmark [11], Kim and Dán used simulations on synthetic and real dependency graphs [10], and Wang et al. combined production data with synthetic datasets containing injected anomalies [7]. Ren et al., in contrast, trained on a fixed dataset of recorded security incidents [8]. The systematic review found the same pattern across its 34 studies: 13 built their own simulator, six used public benchmarks such as the CAGE cyber-defence gyms and the RCAEval root-cause benchmark, and only three used an organisation's operational data [31]. Simulation is thus an accepted response to the scarcity of data, but the available environments model cyberattacks and microservice faults and not the triage of IT service incidents.

The formal tools for representing uncertainty about an incident are well established. A Markov decision process describes a decision problem in which the agent observes the state [4], and a partially observable Markov decision process extends it to the case in which the agent receives only observations and must maintain a belief about the state [5]. Several incident-response studies adopt this view. Hammar et al. estimate a belief with particle filtering [34], Kim and Dán maintain a belief in a hidden Markov model [10], Kovtun et al. track a belief over a hidden attack regime in a security operations centre [30], and Alauthman et al. augment the state

with recent telemetry and alert history to mitigate partial observability [18]. These studies show that a hidden cause with noisy observations is a workable representation of an incident. They are, however, a minority. Of the 34 studies in the systematic review, only four act on an explicit belief, all of them in security settings, and about half treat the incident state as a fully observed vector of metrics or alert features [31].

Taken together, the literature supplies the ingredients for representing ambiguous triage but not the representation itself. It identifies the evidence sources, explains the origins of ambiguity, and offers a formal model for hidden causes. What the reviewed studies do not provide is an environment for IT service incidents in which the level of ambiguity can be set deliberately and in which the available actions are those of triage. The first objective of this study responds to that absence.

2.3 Reinforcement Learning for Investigating, Recommending and Escalating

The second research question concerns how reinforcement learning can support the choice among gathering information, recommending an action and escalating. Research on incident triage has so far framed the task mainly as supervised prediction. The methods surveyed by Remil et al. range from Naive Bayes classifiers over report text to convolutional and recurrent networks that recommend a responsible team [2], and the triage component described by Chen et al. is a recurrent model over the incident text and discussion [1]. One line of this work treats triage as a continuing process and updates its prediction as engineers add to the discussion [1,2]. The model in that case benefits from new evidence but does not decide to seek it. Zhang et al. add that automation in this field mainly stops at triage or the recommendation of solutions [3]. Where agents have been placed in the incident workflow, as in the runbook assistant that D'Antonio and Xie studied over 232 production incidents, the agent guides a human operator who carries out the mitigation [32]. The triage literature therefore predicts well-defined labels but does not model the decision about what to do when the label is uncertain.

Reinforcement learning has entered IT operations and incident handling, mostly for diagnosis and response. Ding et al. learn a policy that prunes a service dependency graph before root cause analysis [35], and Wang et al. use hierarchical reinforcement learning to reduce the number of questions put to engineers when that graph is constructed [7]. In cybersecurity, Ren et al. train an agent to select response strategies [8], Dunsin et al. train an agent to conduct a malware investigation [9], and Alauthman et al. propose a deep Q-network agent for incident response in industrial control systems [18]. Hammar and colleagues treat incident response as planning under partial observability [11, 34]. These studies establish that sequential formulations of incident handling are feasible and can outperform fixed procedures in their own settings. Their action sets, however, consist of responses or investigative steps, and the studies do not address triage as defined in this study.

The decision to gather information has been studied as a sequential problem in its own right. Under partial observability, an action may be chosen for the information it yields, and its value depends on what that information costs [5]. Janisch et al. trained an agent to acquire costly features one at a time before classifying [6]. In incident handling, Kim and Dán choose which alerts to investigate so as to reduce the uncertainty of a belief [10], the agent of Dunsin et al. collects forensic evidence step by step [9], and Wang et al. learn which questions to ask engineers [7]. Information gathering is consequently a tractable learning problem, and one that has already been applied to parts of incident handling. In these studies the alternative to gathering information is to conclude, and none of them offers the agent the further option of

passing the case to a person.

The decision to hand a case to a human has been studied separately, and mostly as a single-step choice. Selective prediction allows a model to abstain when its confidence is low [20], and learning to defer trains a model to pass a case to a human decision-maker by taking that person's ability into account [21, 22]. Strong et al. note that the standard formulation treats cases as independent and optimises a one-step outcome [36]. Sequential extensions exist. Joshi et al. place deferral in a Markov decision process, where deferring affects later states [24], and Zhang et al. formulate deferral as a constrained Markov decision process that accounts for the fatigue of the human expert [37]. Guidance on AI risk management likewise lists deferral to a human expert among the configurations in which an AI system may operate [25]. Applied work treats escalation as an estimate of when a model is likely to be wrong [27] or as a decision that policy can make mandatory [26]. In each case the choice is between acting and deferring, and the option of first gathering more evidence is absent.

The two lines of work thus stop on either side of the choice this study examines. Information-gathering studies offer the agent two options, to look further or to decide, and deferral studies offer two different options, to decide or to hand over. Hammar et al. come closest to joining them: their language-model planner abstains when its candidate actions are inconsistent and then collects external feedback, for example from a digital twin [19]. That system gathers feedback from a model of the system and not from a human, and its decision rule is a consistency threshold and not a learned policy.

Three studies of security alert triage come closer still, and they are the nearest precedents for this study. Sahin and Mert train an agent that queries logs and threat intelligence and then escalates an alert to incident response or closes it, and a separate conformal layer defers the most uncertain alerts to an analyst under certified error budgets [29]. Kovtun et al. learn a policy that chooses among shallow triage, escalation to deep analysis and deferral to gather more context, under constraints on analyst capacity and on missed attacks [30]. Okafor trains offline agents that dismiss, investigate or contain alerts, with escalation to analysts fixed in the deployment design and not learned [38]. These studies show that gathering evidence and handing over to a person can be learned together. None of them, however, recommends a remedy that changes the affected system: their final actions close, prioritise or escalate an alert. The systematic review confirms how rare even this partial combination is. Of its 34 studies, 17 give the agent an explicit information-gathering action, but only seven include escalation to a person, and only Okafor's combines information gathering, a response action and escalation [31]. Among the reviewed studies, none trains a single policy to choose among investigating, recommending a remedy and escalating, and none does so for IT service incidents.

2.4 Rule-Based Safety Mechanisms and Operational Constraints

The third research question asks how effectively a rule-based mechanism can prevent actions that violate operational constraints. Safe reinforcement learning offers two broad ways to impose constraints on a learning agent. García and Fernández divide the field into methods that modify the optimality criterion and methods that modify the exploration process with external knowledge [14]. Constrained policy optimisation belongs to the first group and bounds the expected cumulative cost of a policy [15, 16]. Zhao et al. point out that many practical constraints are instead instantaneous and deterministic and must hold in every state [17]. An operational rule that forbids a particular action under stated conditions is a constraint of this second kind, which an expected-cost bound does not enforce.

Shielding enforces constraints of that kind by separating the rule from the learner. Wen et al. first computed a permissive strategy that satisfies a temporal logic specification and then let the agent learn within it [39]. Alshiekh et al. generalised the idea into a shield that is synthesised from a specification and an abstraction of the environment, and that either restricts the actions offered to the agent or replaces an unsafe action after the agent selects it [12]. Later work relaxed the requirement of absolute safety. Jansen et al. block an action only when it raises the probability of a violation beyond a threshold [40], and Yang et al. use probabilistic logic to handle noisy observations [41]. Hamel-De le Court and colleagues have addressed scalability and uncertainty in the environment model [42, 43]. Könighofer et al. summarise the benefits of shielding as assured compliance with the specification, permissiveness towards safe actions, and preserved convergence of the learner [13].

Shielding has costs that matter for this study. A shield requires a formal model of the safety-relevant part of the environment, and its guarantee extends only as far as that model and the specification are correct [13]. Pranger and Könighofer observe that shielding has seen limited use in practice because it demands expertise in formal methods and considerable engineering effort [44]. Yang et al. note that a shield which simply rejects actions can hinder learning, because the agent is unaware that its choices were overridden [41]. The demonstrations in this literature are grid worlds, arcade games, simulated robots and control tasks [12, 15, 40]. The method is therefore well understood in principle, while its use on the rule sets of IT operations remains largely unexamined.

A parallel body of work enforces explicit rules on language-model agents at run time. Wang et al. define a rule language with triggers, predicates and enforcement actions [45], Xiang et al. and Chen et al. use a second agent to check the actions of the first against stated policies [46, 47], and Pacella et al. place a policy-derived guardrail beneath the agent so that it cannot fall below the escalation that policy requires [26]. Wu and Gong check each proposed action against organisational state and return one of three verdicts, to allow, to block or to ask for clarification [33]. Hu et al. show the scale that rule sets can reach, with a dataset of 12,985 rules drawn from 74 regulations and policies [48]. This work confirms that rules kept outside the decision-maker can be inspected and enforced independently. It concerns language-model agents and not reinforcement learning, and it does not study how enforcement interacts with a policy that is still learning.

Several studies bring rule enforcement into incident response itself. Alauthman et al. couple their deep Q-network agent with an explicit safety shield that enforces operational constraints, in a design where the agent remains advisory by default and the operator can override it [18]. Its setting is cyber-physical control systems, and its agent selects responses without choosing whether to investigate or escalate. The full text reports an evaluation plan and indicative findings shown as directions of change, but no measured values, so it shows that the combination has been proposed and not how it behaves. Hammar et al. check the candidate actions of a language model for consistency with system constraints [19].

Two studies report measurements of a hard rule mechanism. Singh, in a study of remediation decisions in cloud operations, restricts the agent to a feasible region of actions, adds a deterministic guard before execution, and routes an action to a human when the guard fails [49]. Removing the guard left the mean time to resolution almost unchanged, at 144 s against 142 s, but lowered the governance compliance rate from 99.9% to 72.3%. Ravi et al. check every action of an intrusion-response agent for industrial control systems against legal, service-level and safety rules and replace a violating action with a safer alternative or a no-op [50]. Their shield intervened between 526 and 623 times per scenario, and the shielded agent took nine to eleven times more false actions than an unshielded one. The two results point in different

directions, and neither study reports how often blocked proposals occurred over the course of training, which would show whether the agent learned to avoid them.

The systematic review shows how thinly this evidence is spread. Of its 34 studies, eleven express their operational limits only as reward penalties, which discourage a prohibited action without preventing it [13], and eight state no constraint at all. Five use a hard filter or shield, and only three say what happens to a blocked or uncertain decision: substitution with a safer action [50], hand-off to a person [49], or deferral to an analyst [29]. None allows the agent to propose again after a block [31].

The literature therefore supports the feasibility of an independent rule-based mechanism and identifies the questions that remain. It is established that explicit rules can be enforced on a learning agent without preventing it from learning. It is less clear what should replace a blocked proposal when the agent's alternatives include requesting information and escalating, and how often a learned triage policy would propose prohibited actions in the first place. The third objective of this study addresses these questions for IT incident triage.

2.5 Evaluation Measures and Baseline Policies

The fourth research question concerns how the proposed approach should be compared with baseline policies. Each strand of the literature has its own measures. AIOps research evaluates triage with classification and ranking metrics and evaluates the process with time-based measures such as mean time to repair and mean time to engage [2]. Remil et al. criticise the reliance on contingency-table metrics that ignore context and time [2], and Zhang et al. report that generated diagnoses and mitigation suggestions often require human assessment because they lack a fixed ground truth [3]. Reinforcement learning studies of incident response report the effectiveness of the response, the time taken, and false positive rates [8], or the time and accuracy of an investigation [9]. These measures describe whether the final decision was correct and how quickly it was reached.

Studies of constraints and of deferral measure different things. Shielding studies compare a shielded agent with the same agent unshielded and report safety violations alongside reward [12, 40]. Research on language-model agents has introduced measures that combine the two: Levy et al. credit a task only when it is completed without violating any applicable policy, and they find that this measure falls well below the raw completion rate for the agents they test [51]. Mohammadi et al. list compliance with policy among the objectives that agent evaluation in enterprises must cover [52]. Deferral studies report accuracy as a function of coverage [20], the accuracy of the combined human and model [21], or the trade-off between accuracy and the cost of human review [27]. Pacella et al. measure the recall of mandatory escalations and the stability of the decision across runs [26]. Turan relates the escalation rate to realised safety and argues that the relationship need not be monotonic [28].

The baselines used in these studies follow a common pattern. Learned policies are compared with fixed rule-based procedures [8, 26], constrained agents with their unconstrained counterparts [12, 26, 40], and deferring systems with the model alone and the human alone [21, 37]. Dunsin et al. compare their agent with human experts [9]. These comparisons isolate one property at a time, which is the contribution of learning, of the constraint, or of deferral.

The systematic review adds three cautions about these comparisons. First, 20 of its 34 studies compare the learned policy with a static or rule-based one, and in none of the extracted results does the static policy perform better; how much of this reflects the strength of the learned policies and how much the weakness of the baselines cannot be told from the reports [31].

Second, a single metric can hide a degenerate policy. Okafor found that offline agents which had collapsed into trivial detectors still scored well on F1 and ROC-AUC, and that the Matthews correlation coefficient exposed them [38], and the unshielded baseline of Ravi et al. reached an accuracy of 0.98 with a recall close to zero [50]. Third, a simulator can make a policy look better than it is. Sahin and Mert report that an earlier version of their own simulator leaked the label into the confidence signal, so that any threshold appeared to achieve perfect recall [29]. Reporting practice is also weak: only 11 of the 34 studies report results over several runs with a measure of variance, only nine test on conditions not seen in training, and only five report their hyperparameters and release their code or environment [31].

Decision quality, the cost of investigation, escalation behaviour and constraint compliance are each measured somewhere in the literature, but rarely together. Of the 34 studies in the systematic review, 31 measure decision quality and 19 measure time or investigation effort, but only five measure escalation behaviour and four constraint compliance [31]. One study reports all four: Kovtun et al. measure backlog, false escalations, constraint satisfaction and the variance of the policy for escalation in a security operations centre [30]. No study reports all four for the triage of IT service incidents. An agent that can investigate, recommend and escalate could score well on any one of them by neglecting the others, for example by escalating every case or by investigating without limit. The literature thus provides the individual measures and the baseline patterns that this study needs, and it leaves open how they behave when reported together. The fourth objective takes up that question.

2.6 Robustness under Ambiguity and Missing Evidence

The fifth research question asks how robust the approach is as evidence becomes incomplete, conflicting or more ambiguous. The literature treats robustness to imperfect data as a requirement. Remil et al. list it among the properties that AIOps solutions must have [2], and Mohammadi et al. treat robustness to variation as part of the reliability that agent evaluation should probe [52]. Some studies test it directly. Kim and Dán report that their investigation policies remain effective under high false-alert rates and investigation errors [10], and Hammar and Li evaluate response planning when the assumed model of the incident is wrong [11]. Ren et al. acknowledge that their agent generalises less well to attack patterns unlike those in its training data [8]. These results concern the quality of the final decision as conditions degrade.

Uncertainty also affects the mechanisms that are meant to keep an agent within bounds. A deterministic shield assumes that the agent's observations are accurate, an assumption Yang et al. describe as unrealistic [41], and a shield computed from an estimated model can lose its guarantee when the estimate is wrong [43]. Wu and Gong show that a rule cannot be applied correctly when the facts it depends on are hidden from the checker [33]. A rule-based mechanism in triage faces the same difficulty, because a rule whose condition refers to the state of a system can only be evaluated on the evidence available.

The human side of escalation is not constant either. Learning when to defer depends on data about the expert that is costly to obtain [22], people decide differently when they know that a model has deferred to them [23], and expert accuracy changes with workload [37]. Turan argues from a modelling study that increasing the escalation rate can reduce realised safety once the reviewer is overloaded [28]. An agent that responds to ambiguity only by escalating more would therefore transfer the problem and not resolve it.

The systematic review found robustness tests in 15 of its 34 studies, under model error, system change, unseen attackers, weaker detection and dataset shift, and the methods that keep a belief or adapt their model degraded least [11, 31, 34]. One result bears directly on escalation. Sahin and Mert show that when the scoring model degrades under dataset shift, the share of alerts deferred to analysts rises, to between 79% and 99% for the most affected classifiers, so that failure becomes visible instead of silent [29]. That result concerns one decision, deferral, as the scoring model degrades. The reviewed studies have not examined how the balance among investigating, recommending and escalating shifts as ambiguity rises. Robustness has been measured for the accuracy of a decision, for the validity of a shield, and for the reliability of a human reviewer, each in isolation. For a triage agent, the behaviour of interest is the change in its mix of decisions: whether it investigates more, escalates more, or continues to recommend with less support. The fifth objective of this study measures that change across controlled levels of ambiguity and missing evidence.

2.7 Synthesis of the Reviewed Literature

Across the five themes the literature shows a consistent pattern: the components that this study needs exist, but they have been developed and evaluated apart. AIOps research describes the triage task and its evidence but models it as prediction. Reinforcement learning research on incident handling models sequential decisions but concentrates on response and investigation in cybersecurity. Shielding research provides a way to enforce rules on a learning agent but has been demonstrated mainly on control tasks. Deferral research explains when a model should hand a decision to a person but treats the hand-over mostly as a single-step alternative to prediction.

Table 2.1 compares the studies closest to this one against the capabilities that the research questions concern. The comparison shows that each capability appears in at least one study and that the studies cluster in pairs of capabilities. Sequential policies are combined with information gathering [7,9,10], or with a rule check [18,50], or with deferral [37]. Rule checks are combined with escalation where the decision-maker is a language model [26]. The three studies of security alert triage combine information gathering with escalation, but their constraints are statistical budgets or a severity floor and not rule checks on a remedy [29, 30, 38]. The one study of IT service operations with a rule check and a hand-off to a person, Singh's, neither gathers information nor learns when to escalate; its hand-off is triggered by the rule check [49]. No row other than the proposed study combines a sequential policy, information gathering, a rule check and learned escalation.

The comparison also qualifies any claim to novelty. A reinforcement learning agent with a safety shield has been proposed for incident response in industrial control systems [18], and another evaluated in the same setting [50], a deterministic guard with a hand-off to a person has been evaluated for remediation in cloud operations [49], agents that gather evidence and escalate or defer have been evaluated for security alert triage [29,30], and a planner that checks its actions against constraints and seeks further feedback when uncertain has been evaluated for security incidents [19]. This study does not introduce the idea of constraining a learned incident-handling agent. Its contribution lies in the particular decision it studies, the setting in which it studies it, and the way it evaluates the result.

Table 2.1: Comparison of the studies closest to this research. ✓ = addressed; ∘ = partly addressed; – = not addressed or not reported. Studies marked † were assessed from their abstracts.

Study	Setting	*Sequential policy*	*Information gathering*	*Rule check*	*Human escalation*	*Varied ambiguity*
Chen et al. [1]	IT incident triage	–	–	–	–	–
Wang et al. [7]	Root cause analysis	✓	✓	–	∘	–
Ren et al. [8]	Security incident response	✓	–	–	–	–
Dunsin et al. [9]	Malware investigation	✓	✓	–	–	–
Kim and Dán [10]†	Security alert prioritisation	✓	✓	–	–	✓
Hammar and Li [11]	Security incident response	✓	–	–	–	✓
Hammar et al. [19]†	Security response planning	∘	∘	✓	–	–
Alauthman et al. [18]	Industrial control incident response	✓	–	✓	∘	–
Ravi et al. [50]	Industrial control incident response	✓	–	✓	–	–
Okafor [38]	Security alert triage	✓	✓	∘	∘	–
Sahin and Mert [29]	Security alert triage	✓	✓	∘	✓	✓
Kovtun et al. [30]	Security alert escalation	✓	∘	∘	✓	–
Singh [49]	Cloud operations remediation	∘	–	✓	∘	–
Zhang et al. [37]	Classification with deferral	✓	–	–	✓	–
Pacella et al. [26]	Employee onboarding workflow	–	–	✓	✓	–
Wu and Gong [33]	Workplace assistant actions	–	∘	✓	–	–
This study (proposed)	IT incident triage	✓	✓	✓	✓	✓

2.8 Research Gap

The review identifies five related gaps, one under each research question. The first is a gap in representation. The reviewed studies explain why triage evidence is ambiguous and provide formal models for hidden causes, but they offer no controlled environment for IT service incidents in which ambiguity can be varied and in which the actions are those of triage. The available environments model cyberattacks and microservice faults [7, 9, 11], and only four of the 34 studies in the systematic review represent the hidden state of an incident with an explicit belief [31].

The second gap concerns the decision itself. Information-gathering studies let an agent choose between looking further and deciding [6, 10], and deferral studies let it choose between deciding and handing over [21, 24, 37]. Studies of security alert triage have joined the two, learning to gather evidence and to escalate or defer an alert [29, 30], but their agents do not recommend a remedy that changes the affected system. No reviewed study trains one policy to choose among investigating, recommending a remedy and escalating, and none addresses IT service incidents in this way. It is consequently unknown whether an agent can learn to treat investigation and escalation as distinct responses to uncertainty when one of its options acts on the system, or whether it will collapse into one of them.

The third gap concerns the interaction between rules and that decision. Rule-based shields have been shown to restrict learned policies in control tasks [12, 13], and hard rule mechanisms have been measured in industrial incident response and cloud remediation [49, 50]. Only three of the 34 studies in the systematic review report what happens to a blocked decision, none lets the agent propose again, and none reports how often blocked proposals occur over training [31]. The reviewed literature does not report how an independent rule-based mechanism behaves when the constrained agent can also request information or escalate.

The fourth and fifth gaps concern evidence. Decision quality, investigation cost, escalation behaviour and constraint compliance have been measured together for one agent in a single study, of escalation in a security operations centre [30], and not for the triage of IT service incidents, so the trade-offs among them have not been observed in that setting. Robustness has been tested for decision accuracy, for the validity of shields and for the rate of deferral [10, 11, 29, 43], but not for the mix of investigating, recommending and escalating as ambiguity increases.

This study addresses these gaps within stated limits. It will construct a simulated environment for ambiguity-aware IT incident triage, train a rule-constrained reinforcement learning agent to choose among gathering diagnostic information, recommending a permitted action and escalating to a human expert, and evaluate the agent against baseline decision policies on all four measures and across levels of ambiguity. The gaps are stated for the literature reviewed in this chapter. Related work may exist beyond it, and the claim made here is that the reviewed literature leaves these questions open, not that they have never been examined.

2.9 Chapter Summary

This chapter reviewed the literature under five themes that correspond to the research questions of the study. It showed that the literature explains the sources of ambiguity in triage and supplies formal models for it, that sequential decision methods have been applied to incident response and investigation but not to the three-way choice of triage, that rule-based mechanisms can constrain learning agents but have rarely been examined in IT operations, and that evaluation measures and robustness tests exist separately for each strand. A comparison

of the closest studies, including those identified by a systematic review of 34 studies, showed that security alert triage has combined information gathering with escalation and that rule mechanisms have been measured in industrial and cloud settings, but that none combines a sequential policy, information gathering, a rule check and learned escalation, and the chapter stated five gaps on that basis. Chapter Three presents the methodology by which the study addresses them.

Chapter 3

Methodology

This chapter describes how the study will be carried out. It presents the research design and the pipeline of the project, and then describes each component in turn: the modelling of incident scenarios, the simulated triage environment, the reinforcement learning agent, the rule-based safety mechanism and the baseline decision policies. It then defines the evaluation metrics and the experiments, and closes with the tools, the threats to validity and the ethical considerations of the study. The chapter describes planned work, and it reports no results.

3.1 Research Design

The study follows a design-and-evaluate approach. It constructs an artefact, a rule-constrained reinforcement learning agent together with the environment in which the agent operates, and it evaluates that artefact through controlled experiments. This corresponds to the design science view of research in information systems, in which knowledge is produced by building an artefact that addresses an identified problem and by evaluating it rigorously [53]. The design suits the research questions, which ask how a task can be represented, how a method can be applied to it, and how well the result performs.

The evaluation is experimental and quantitative, and it is conducted entirely in simulation. Chapter Two showed that realistic incident data is scarce and confidential and that simulation is the accepted response in studies of sequential decision-making for incident handling [2, 9, 11]. Simulation also gives the study control over the variable of interest. The level of ambiguity can be set, held constant and varied, which would not be possible with live incidents. The cost of this choice is that the findings describe the behaviour of the agent under the assumptions of the simulator, a limit that Section 3.11 discusses.

A qualitative component informs the simulation. The Directorate of University ICT Services at Uganda Christian University has agreed to support the study, and its practice is the source from which the incident scenarios and operational rules are drawn. An initial semi-structured interview with a senior developer of the Directorate, held on 19 October 2026, described the current handling of incidents: users or support staff raise tickets, which are vetted and assigned to the responsible developer or, where that person is not known, to a senior developer; application errors are observed through system logs and a monitoring dashboard. The interviewee identified the application programming interface logs as the most informative source of evidence. This consultation shaped the incident categories in Section 3.3 and the treatment of escalation as a hand-over to the responsible member of staff. Table 3.1 shows how each specific objective is addressed by a component of the method and by a set of measures.

Table 3.1: Alignment of the specific objectives with the method and the evaluation.

Objective	Method component	Evidence produced
1. Identify and model ambiguous triage scenarios	Scenario elicitation and incident model (Sections 3.3 and 3.4)	Scenario model reviewed by Directorate staff
2. Design the reinforcement learning environment	Simulated triage environment and agent (Sections 3.4 and 3.5)	Working environment; learning curves
3. Implement the rule-based safety mechanism	Safety mechanism (Section 3.6)	Constraint compliance measures
4. Evaluate against baseline policies	Baselines, metrics and Experiments 1 to 3 (Sections 3.7 to 3.9)	Decision quality, investigation efficiency, escalation behaviour, constraint compliance
5. Assess robustness under ambiguity	Experiment 4 (Section 3.9)	All measures across ambiguity levels

3.2 Research Pipeline

The project proceeds through eight phases, shown in Figure 3.1. The first phase, the review of the literature and the specification of the problem, is reported in Chapters One and Two. The second phase elicits incident scenarios and operational rules with the Directorate. The third phase builds the simulated environment and tests it, and the resulting scenarios are returned to Directorate staff for review before they are fixed. The fourth and fifth phases implement the safety mechanism and the baseline policies. The sixth phase trains the agents, the seventh runs the experiments, and the eighth analyses and reports the results.

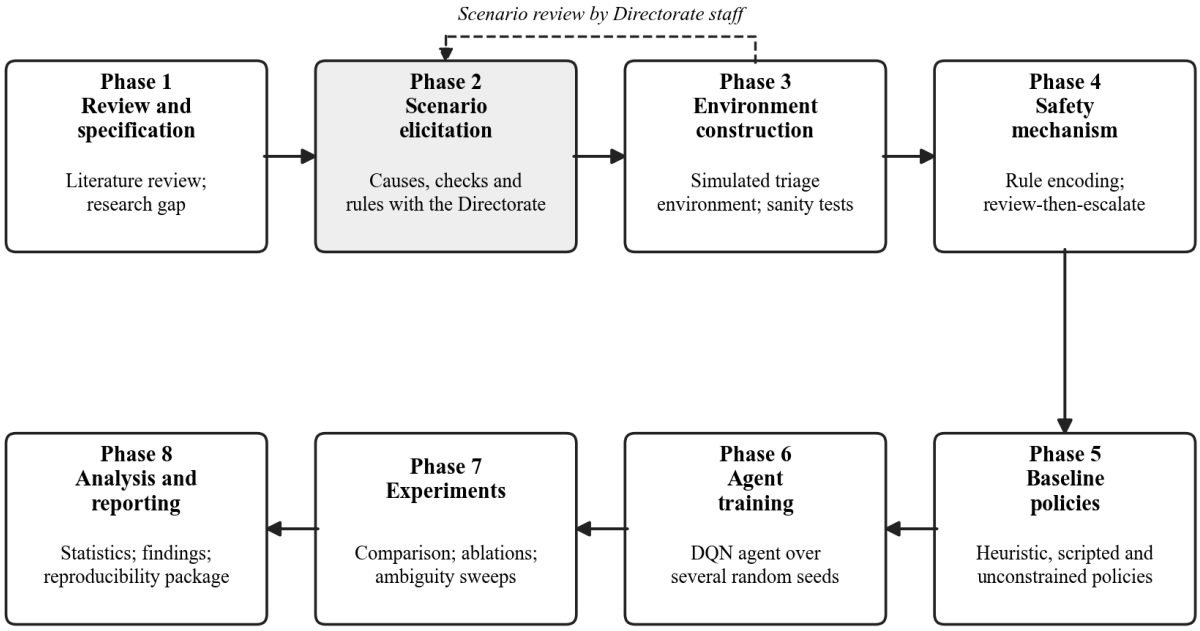


Figure 3.1: The research pipeline of the study.

The order of the phases reflects their dependencies. The environment cannot be built before the scenarios are specified, and the rules cannot be encoded before the actions and evidence they refer to exist in the environment. The baselines are implemented before the agent is trained so

that the environment is exercised by simple policies first, which exposes errors in the simulator before they can be mistaken for properties of the learned agent.

3.3 Scenario Modelling with the Directorate

The scenario model defines what an incident is in the simulation. It consists of about 15 causes, 20 diagnostic checks and 10 operational rules. Fifteen causes is a target, not a limit: more are added if the Directorate can describe them well, since a smaller model that practitioners can rate and review with care is preferred to a larger one whose later entries are guesses. Following the initial consultation, the causes are drawn mainly from application errors observed in the logs of the university's systems, together with infrastructure causes that surface in those logs. They are distributed across four provisional categories: application and interface errors, authentication and access failures, database failures, and failures of infrastructure and external services. The categories are confirmed or revised in the elicitation. Each cause is associated with one remedy, which is the action that resolves an incident with that cause. Each check is a diagnostic observation that a member of IT staff could obtain, such as an entry in the application logs, a value on the monitoring dashboard, the result of reproducing a failing request or the state of a service, and each check has a cost that represents the effort of obtaining it. The symptoms correspond to what is first seen of an incident, such as the type of error and the affected function recorded in the logs, or the category of a user's ticket.

The content of the model is elicited from the Directorate. The elicitation seeks the causes of incidents that the Directorate encounters in these categories and the remedy for each, the symptoms with which those incidents are first reported, the checks that IT staff use to distinguish one cause from another, the rules that govern which actions may be taken and under what conditions, and the attributes of an incident to which those rules refer, such as whether the affected system is in production and the priority of the incident. The symptoms form a fixed list, so that each symptom is a named indicator and not free text. For each pair of cause and check, and for each pair of cause and symptom, the elicitation seeks a judgement of how likely the check or symptom is to be positive when that cause is present. It also seeks the approximate relative frequency of each cause and the effort required by each check. Where the Directorate can offer them, it seeks the relative costs of an escalation and of a wrong remedy and the frequency with which checks are unavailable or misleading; these inform the reward values and the ranges of the ambiguity controls but do not fix them. Any parameter that the elicitation does not supply is set by the researcher and reported as such.

The likelihoods are elicited on a five-point verbal scale, from *never* through *rarely*, *sometimes* and *often* to *almost always*, which is mapped to the probabilities 0.05, 0.25, 0.5, 0.75 and 0.95. Asking for a verbal rating rather than a probability for each of the 300 pairs of cause and check keeps the task manageable for practitioners. Judgements are sought for the pairs that the Directorate considers related, and unrelated pairs take a default value that is reported with the model. The source of each judgement is recorded. These judgements become the parameters of the incident model described in Section 3.4. The arrangements for the elicitation, including whether it draws on consultation with IT staff, on anonymised counts from the Directorate's ticketing and monitoring systems, or on both, are subject to the agreement reached with the Directorate. Such counts, for example of incidents by type and of the time taken to resolve them, would inform the relative frequency of each cause and the cost of an escalation.

The model is validated by expert review. After the environment has been built, sample incidents generated by the simulator are presented to Directorate staff, who judge whether the combinations of symptoms, check results and causes are plausible and whether the rules are stated

correctly. The model is revised in response and then fixed before any agent is trained. Fixing the model before training prevents the scenarios from being adjusted, even unintentionally, in favour of the proposed agent.

The scenario model is a simplification, and the study states its simplifying assumptions. Each incident has exactly one cause, each cause has one correct remedy, and the result of a check does not change during an incident. Symptoms reach the agent as structured indicators from the elicited list; extracting them from the free text of a ticket is outside the scope of the study. These assumptions keep the model small enough to be specified and reviewed by people, at the cost of excluding incidents with several interacting causes and the errors that would arise in reading symptoms from text.

3.4 The Simulated Triage Environment

The environment casts triage as a sequential decision problem under partial observability. Chapter Two showed that a hidden cause with noisy observations is an established representation of an incident [5, 10, 34]. The environment follows that representation and is implemented with the standard interface for reinforcement learning environments [54]. One episode of the environment corresponds to the triage of one incident.

3.4.1 Incident Model

At the start of an episode the environment draws a cause c from the set of causes $C = \{c_1, \ldots, c_K\}$, where K is the number of elicited causes (about 15), according to a prior distribution $P(c)$. The cause is hidden from the agent for the whole episode. The environment also draws a context for the incident, consisting of attributes such as whether the affected system is a production system and the priority of the incident. The context is visible to the agent, and the operational rules refer to it.

Evidence about the cause is generated by a likelihood model. Each diagnostic check d_i in the set $D = \{d_1, \ldots, d_{20}\}$ has a binary result, and the probability of a positive result depends on the cause:

$$\theta_{ij} = P(d_i = \text{positive} \mid c_j). \tag{3.1}$$

The initial symptoms of the incident are generated in the same way from a set of symptom indicators, and they are revealed to the agent at the first step. The true result of every check is drawn once at the start of the episode, independently of the other checks given the cause, so that requesting a check twice returns the same result.

3.4.2 Observations and State

The agent never observes the cause. It observes an evidence vector that records, for each check, one of four values: not yet requested, unavailable, negative or positive. Each symptom indicator takes one of three values, present, absent or not reported, since all symptoms are revealed at the first step. The vector also contains the context attributes, the number of steps taken, and an indicator that is set when the previous proposal was blocked by the safety mechanism. This evidence vector is the agent's estimate of the incident state. Giving the learner access to the state of the safety mechanism follows Alshiekh et al., whose learner can base its actions on the state of the shield [12].

The evidence vector is presented to the agent as a fixed-length vector of numbers. Each categorical value, namely the result of a check, the value of a symptom and each context attribute

with more than two levels, is one-hot encoded, so that no artificial ordering is imposed on its values. Binary attributes and the blocked indicator take the values 0 and 1, and the number of steps is divided by the step limit so that it lies between 0 and 1. The length of the vector is fixed by the number of symptoms and context attributes established in the elicitation.

Because the results of checks do not change within an episode, the vector summarises everything the agent has learned about the incident, and the agent does not need a memory of earlier steps. Studies of incident response in which evidence arrives as a stream of events encode a history, with a transformer over recent security events [8] or with recent telemetry and alert history [18], and a model of a fatiguing human expert uses a sequence model to track workload over an episode [37]. The static evidence assumed here allows a feed-forward network to be used instead.

This choice does not give up the belief that a partially observable formulation requires. In the systematic review, the methods that kept an explicit belief over the hidden state were the most robust to model error and system change [11, 31, 34]. In this environment the results of checks are fixed within an episode and independent of one another given the cause, and missing and reversed results arise independently of the cause. The posterior probability of each cause is then determined by the prior, the initial symptoms and the results observed so far, so the evidence vector is a sufficient statistic for the belief that a partially observable agent would maintain [5]. The agent does not compute that belief explicitly, because it is not given the likelihood model; it must learn the mapping from evidence to action. The threshold policy of Section 3.7, which is given the likelihood model and computes the posterior directly, serves as the explicit-belief reference.

3.4.3 Actions

The agent has $20+K+1$ actions, 36 with 15 causes. Twenty actions each request one diagnostic check, K actions each recommend the remedy for one cause, and one action escalates the incident to a human expert. A request returns the result of the check and continues the episode. A check that has already been requested is removed from the agent's choices for the rest of the episode, since repeating it would return the same result. A recommendation or an escalation ends the episode. An episode also ends, as an escalation, if the agent reaches a fixed limit on the number of steps without deciding.

3.4.4 Reward

The reward expresses the costs that Chapter One attributed to each choice. A correct recommendation receives a positive reward r_{correct}. A wrong recommendation receives a penalty $-\lambda_{\text{wrong}}$. Each request for check d_i incurs its cost $-k_i$. An escalation incurs a cost $-\lambda_{\text{esc}}$, which represents the expert's time. The values satisfy

$$r_{\text{correct}} > -\lambda_{\text{esc}} > -\lambda_{\text{wrong}}, \tag{3.2}$$

so that resolving an incident correctly is better than escalating it, and escalating it is better than recommending the wrong remedy. The simulated expert is assumed to resolve every escalated incident. The numerical values are set in pilot runs before the experiments begin and are reported with the results. Because the behaviour of the agent depends on these values, the study also examines how the results change when the cost of escalation is varied.

3.4.5 Ambiguity Controls

Chapter Two identified three sources of ambiguity: missing observations, unreliable observations, and observations consistent with more than one cause. The environment has one control for each.

1. **Missing evidence.** With probability p_{miss}, a check is unavailable for the whole episode, and a request for it returns the value "unavailable".

2. **Unreliable evidence.** With probability p_{noise}, the result of a check is reversed before the agent receives it.

3. **Overlapping evidence.** An overlap parameter ω between 0 and 1 moves the likelihood profile of each cause towards the average profile of its category, $\theta'_{ij} = (1 - \omega)\,\theta_{ij} + \omega\,\bar{\theta}_i$, where $\bar{\theta}_i$ is the mean of θ_{ij} over the causes in the same category as c_j. As ω increases, causes within a category become harder to tell apart.

A setting of the three controls defines an ambiguity level. The base level, used for training and for the main comparison, is fixed in pilot runs so that the task is neither trivial nor impossible. The planned ranges for the robustness experiment are $p_{\text{miss}} \in \{0, 0.2, 0.4, 0.6\}$, $p_{\text{noise}} \in \{0, 0.1, 0.2, 0.3\}$ and $\omega \in \{0, 0.25, 0.5, 0.75\}$.

3.4.6 Reference Labels

The simulator knows the hidden cause and the likelihood model, so it can label each episode with the outcome that would have been appropriate. An incident is labelled as one that should be escalated in either of two cases. In the first, the operational rules prohibit the agent from recommending the remedy for the true cause in the context of the incident. In the second, the incident is irreducibly ambiguous: even with the result of every available check, the most probable cause under the simulator's own model is not the true cause. All other incidents are labelled as resolvable. These labels are used only for evaluation and are never shown to the agent.

The simulator is checked for leakage of the hidden cause before any agent is trained. Sahin and Mert found that an earlier version of their own simulator leaked the label into the agent's observations, so that any threshold appeared to achieve perfect recall [29]. In this environment no policy that sees only the observations can, on average, resolve correctly a larger share of incidents than the recommendation of the most probable cause after every available check, and by the definition above that share is the share of incidents labelled as resolvable. This ceiling is computed at each ambiguity level, and a learned or baseline policy whose resolution accuracy exceeds it by more than sampling error is taken as evidence of leakage, to be corrected before the experiments proceed.

3.5 The Reinforcement Learning Agent

The agent learns a policy that maps the evidence vector to one of the $20 + K + 1$ actions. The primary learning algorithm is the deep Q-network (DQN), which estimates the expected return of each action with a neural network [55]. Three considerations motivated the choice. First, the action set is small and discrete, which value-based methods handle directly. Second, the evidence vector admits 4^{20} combinations for the checks alone, which rules out the tabular Q-learning used in earlier incident-handling and shielding studies [9, 12] while retaining the same family of methods. Third, DQN yields an estimated value for every action, so

the values the agent assigns to a blocked remedy and to escalation can be inspected when its behaviour is analysed. Constrained optimisers such as constrained policy optimisation and PPO-Lagrangian [15, 37] were not adopted, because they bound the expected cumulative cost of a policy, whereas the operational rules must hold at every step and are enforced by a separate mechanism (Section 3.6). Policy-gradient methods are common in recent shielding studies [13, 41], and proximal policy optimisation [56] therefore serves as a secondary learner. Where time permits, the main comparison is repeated with it to show whether the findings depend on the choice of learner. Both algorithms are taken from an established open-source library to limit implementation error [57]. The library's DQN does not support action masking, so the removal of blocked actions and of checks already requested is implemented by restricting the choice of action, and the maximum in the learning target, to the available actions.

Each agent is trained by interaction with the environment at the base ambiguity level. Training begins from the library's default hyperparameters, and a small search over the learning rate, the network size and the exploration schedule is carried out on a set of random seeds that is not used in the final experiments. The training length is set from the pilot runs as the point at which the mean return stops improving. The final hyperparameters, the training length and the search itself are reported with the results.

A second training regime is used for the robustness question. In it the ambiguity controls are drawn at random from their planned ranges at the start of each episode, so that the agent meets many levels of ambiguity during training. Comparing the two regimes shows whether robustness requires exposure to ambiguity during training or carries over from training at a single level.

3.6 The Rule-Based Safety Mechanism

The safety mechanism is a component separate from the agent that checks every proposed action against the operational rules. Each rule is a condition over three things: the proposed action, the context of the incident, and the evidence gathered so far. A rule may, for example, prohibit a remedy on a production system, or prohibit a remedy until a named check has been observed. An action is prohibited if any rule prohibits it. Escalation is never prohibited, so a permitted action always exists. The ten rules are those elicited from the Directorate, encoded as explicit conditions that can be read and reviewed without reference to the agent.

The mechanism follows a review-then-escalate procedure when it blocks a proposal.

1. The agent proposes an action. If no rule prohibits it, the action passes to the environment unchanged.

2. If a rule prohibits the action, the mechanism blocks it and records the event. The step is consumed without reward. The agent receives the same evidence vector with the blocked indicator set, and the blocked action is removed from its choices for the next step.

3. The agent proposes a second action. If it is permitted, it passes to the environment and the episode continues as normal.

4. If the second proposal is also prohibited, the mechanism ends the episode by escalating the incident to the human expert. This outcome is recorded as a forced escalation.

This design responds to three findings of Chapter Two. The first is that its central step has no precedent: of the 34 studies in the systematic review, only three report what replaces a blocked decision, by substitution, hand-off or deferral, and none gives the agent a second

proposal [29, 31, 49, 50]. The other two concern how the block is handled. A shield that silently replaces an action can hinder learning because the agent does not know that it was overridden [41]; here the agent is told of the block and given one opportunity to choose again. Where a decision cannot be made within the rules, the appropriate fallback is to hand it to a person [26]; here a second block has that effect. The mechanism belongs to the family of shields that act after the agent has chosen [12]. It differs from them in that its rules are written directly and not synthesised from a formal model of the environment, so it carries no formal guarantee beyond the rules as written. Within the simulation it ensures that no prohibited action is executed.

3.7 Baseline Decision Policies

No published system can be compared with the proposed agent directly, because the closest studies address other domains and share no environment with this one. The agent is therefore compared with baseline policies that run in the same environment, each standing for an approach identified in Chapter Two. Table 3.2 lists them.

Table 3.2: Baseline decision policies and the approaches they represent.

Policy	Behaviour	Approach represented
Initial-symptom policy	Recommends the most probable cause given the initial symptoms, without requesting checks	Triage as a single prediction [1, 2]
Scripted policy	Requests a fixed sequence of checks for the reported category and then recommends the most probable cause	Rule-based and runbook practice [2, 32]
Always-escalate policy	Escalates every incident	The limiting case of deferral
Threshold policy	Maintains the posterior probability of each cause, requests the most informative check while no cause exceeds a confidence threshold, recommends when one does, and escalates when the checks are exhausted	Selective prediction and uncertainty-driven investigation [10, 20]
Unconstrained agent	The same learning agent trained and run without the safety mechanism	The standard comparison in shielding studies [12, 40]
Penalty-only agent	The same learning agent without the mechanism, with a reward penalty for each prohibited action	Constraint by reward penalty [13]

The order of checks in the scripted policy is elicited from the order in which Directorate staff examine evidence for each category of incident, so that the policy reflects current practice and not an arbitrary sequence. This matters because, in the systematic review, no extracted result showed a static baseline outperforming a learned policy, and weak baselines could not be ruled out as the reason [31]. Where the elicitation does not supply an order for a category, the checks are ordered by their expected information about the cause under the elicited likelihoods, and this is reported.

Two of these baselines deserve comment. The threshold policy is given the simulator's likelihood model, which the learning agent must infer from experience. It is therefore a strong reference,

and the study does not assume that the learned agent will exceed it. The always-escalate policy executes no prohibited action and makes no wrong recommendation, and it shows why no single measure can be read alone. The first four policies are run with the safety mechanism in place, so that their proposals are checked in the same way as those of the proposed agent.

Two ablations of the proposed agent complete the comparison. In the first, the agent cannot request checks and must recommend or escalate from the initial symptoms, which corresponds to the two-way choice studied in the deferral literature [21]. In the second, the agent cannot escalate voluntarily and must gather evidence and then recommend, which corresponds to the two-way choice studied in the literature on information acquisition [6]. The ablations show what each option contributes to the three-way decision.

3.8 Evaluation Metrics

Every policy is evaluated on the same fixed set of N evaluation episodes at each ambiguity level. Each episode ends in a correct recommendation, a wrong recommendation or an escalation. Let N_{c}, N_{w} and N_{e} be the numbers of episodes with each outcome, so that $N = N_{\mathrm{c}} + N_{\mathrm{w}} + N_{\mathrm{e}}$. The metrics are standard measures taken from the strands of literature reviewed in Chapter Two, and they are grouped under the four measures named in the fourth objective.

3.8.1 Decision Quality

Decision quality describes how often the agent's final decision is right. *Resolution accuracy* is the share of all incidents that the agent resolves itself, N_{c}/N. It is the usual accuracy of the triage literature [2], and it penalises an agent that avoids error by escalating. *Coverage* is the share of incidents on which the agent makes a recommendation, $(N_{\mathrm{c}} + N_{\mathrm{w}})/N$, and *selective accuracy* is the accuracy of those recommendations, $N_{\mathrm{c}}/(N_{\mathrm{c}} + N_{\mathrm{w}})$. These two are the standard pair of selective prediction [20]. They are reported together because either can be raised at the expense of the other. The *wrong-recommendation rate*, N_{w}/N, isolates the most costly error in triage. The *mean episodic return* is the average total reward per episode. It is the quantity the agent is trained to maximise and the standard summary in reinforcement learning [4], but it depends on the reward values chosen and is never reported alone.

3.8.2 Investigation Efficiency

Investigation efficiency describes what the agent spends on evidence before it decides. The *mean number of checks* is the average number of requests per episode, and the *mean investigation cost* is the average of the summed check costs per episode. The cost stands in for time, which a simulation without a clock cannot measure directly. The two are read together with accuracy as an *accuracy–cost curve*, following the evaluation of costly feature acquisition [6]. The curve shows whether additional investigation yields additional accuracy and allows policies to be compared at equal cost.

3.8.3 Escalation Behaviour

Escalation behaviour describes how much the agent hands to the human expert and whether it hands over the right cases. The *escalation rate* is N_{e}/N, the load placed on human experts [28]. The remaining metrics use the reference labels of Section 3.4.6. *Escalation precision* is the share of escalated incidents that were labelled as ones that should be escalated; low precision means that human experts receive incidents the agent could have resolved. *Escalation recall*

is the share of incidents labelled as ones that should be escalated that the agent did escalate; low recall is the failure of under-escalation that Pacella et al. measured [26]. The *escalation F1 score* is the harmonic mean of the two. Because precision, recall and F1 can all look acceptable when one class is rare, the *escalation Matthews correlation coefficient* between the agent's choice to escalate and the reference label is also reported; it uses all four cells of the confusion matrix, and Okafor found that it exposed degenerate triage policies that F1 and ROC-AUC concealed [38]. Forced escalations are excluded from these three metrics and reported under constraint compliance, because they are produced by the mechanism and not chosen by the agent. Escalations caused by the step limit are likewise counted separately.

3.8.4 Constraint Compliance

Constraint compliance describes the agent's relationship to the rules. The *proposed-violation rate* is the number of blocked proposals divided by the number of proposals. It shows whether the agent has learned to act within the rules or relies on being stopped. The *forced-escalation rate* is the share of episodes that end with a second blocked proposal. The *executed-violation rate* is the share of episodes in which a prohibited action takes effect. It is zero by construction when the mechanism is active, and it is the central measure for the unconstrained and penalty-only agents, as in the shielding literature [12, 40]. The *compliant success rate* is the share of episodes that end in a correct recommendation with no executed violation. It follows the measure of completion under policy introduced for agent benchmarks [51], and for agents without the mechanism it shows how much of their accuracy was obtained by breaking rules.

3.8.5 Robustness

Robustness is not a separate metric. Each metric above is computed at every ambiguity level and reported as a function of that level, in the manner of studies that vary false-alert rates or model error [10, 11]. Two views are reported. The first is degradation, the change in resolution accuracy and wrong-recommendation rate relative to the base level. The second is the decision mix, the coverage, the escalation rate and the mean number of checks at each level, which shows whether the agent responds to ambiguity by investigating more, by escalating more, or by continuing to recommend on weaker evidence.

3.9 Experimental Design

The study comprises four experiments. Each learning agent is trained with ten random seeds, and results are reported as the mean over seeds with a 95% bootstrap confidence interval, following recommended practice for reporting reinforcement learning results [58]. The evaluation episodes are generated with seeds that are not used in training, and the same episodes are presented to every policy so that differences between policies are not due to differences between incidents. These choices answer the reporting weaknesses found in the systematic review, in which only 11 of 34 studies reported results over several runs with a measure of variance, nine tested their agents on conditions not seen in training, and five reported their hyperparameters and released their code or environment [31]. This study reports variance over seeds, evaluates on unseen episodes and on ambiguity levels not used in training, and archives its code and configurations (Section 3.10).

Experiment 1: comparison with baselines. The proposed agent is compared with the six baseline policies at the base ambiguity level on all metrics. This experiment addresses the fourth research question and the first hypothesis.

Experiment 2: effect of the safety mechanism. The proposed agent is compared with the unconstrained agent and the penalty-only agent on the constraint compliance metrics and on decision quality, and the proposed-violation rate is tracked over training. The two studies in the systematic review that measured a hard rule mechanism reached opposite results, one finding that the rules cost almost nothing [49] and the other that they produced many false actions [50], and neither reported how often blocked proposals occurred as training progressed. Tracking that rate shows whether the agent learns to act within the rules or continues to rely on being stopped. This experiment addresses the third research question and the second hypothesis.

Experiment 3: ablation of the action set. The proposed agent is compared with its two ablations to establish what the options to investigate and to escalate each contribute. This experiment addresses the second research question.

Experiment 4: robustness. All policies are evaluated across the planned ranges of the three ambiguity controls, varied one at a time from the base level and then jointly, for agents trained under both training regimes. This experiment addresses the fifth research question and the third hypothesis.

The first research question, on representation, is answered by the scenario model and the environment themselves and by their review by Directorate staff. A sensitivity analysis accompanies the experiments: the cost of escalation is varied around its chosen value and Experiment 1 is repeated, to show how far the conclusions depend on the reward design.

3.10 Tools and Implementation

The environment, the safety mechanism and the baselines are implemented in Python. The environment follows the Gymnasium interface [54], and the learning algorithms are those of Stable-Baselines3 [57]. The scenario model, the rules and the settings of every experiment are held in configuration files and not in code, so that each result can be traced to the configuration that produced it. Random seeds are fixed and recorded. The code, the configurations and the evaluation episodes will be archived with the project so that the experiments can be repeated, with any material that the Directorate regards as confidential withheld or generalised.

3.11 Validity and Limitations

The main threat to the validity of the study is the realism of the simulator. The agent's behaviour is valid for incidents generated by the likelihood model, and that model rests on elicited judgements and on the simplifying assumptions of Section 3.3. Expert review of generated incidents reduces this threat but does not remove it. The findings should therefore be read as evidence about the method under stated assumptions, and not as a prediction of performance on live incidents.

Three further limitations follow from design choices. First, the simulated expert resolves every escalated incident at a fixed cost. Chapter Two showed that the accuracy of human reviewers varies with workload [28, 37], so the study understates the cost of heavy escalation, and it reports the escalation rate so that this cost remains visible. Second, the results depend on the reward values, which the sensitivity analysis addresses only for the cost of escalation. Third, the safety mechanism enforces the rules as written, and it can only evaluate a rule on the evidence available; a rule that depends on a fact the agent has not observed cannot be applied correctly [33].

The study also limits what it claims about generality. The scenario model reflects the practice of one organisation and four categories of incident, and the initial consultation drew on a single interview. Whether the findings hold for other organisations, other categories or larger models is a question for further work.

3.12 Ethical Considerations

The study will be conducted with the permission of the Directorate and in accordance with the research ethics requirements of the university. Participation by Directorate staff in the interviews, elicitation and review is voluntary, consent is obtained before any statement is used, and no member of staff is identified in the report. The initial consultation identified the security of incident data, including control over who can see it, as the main condition for using an agent of this kind; this is one reason the study works from elicited parameters in simulation and not from live data. Any incident records used to inform the scenario model are anonymised before use, and no personal data, credentials or confidential details of the university's systems appear in the simulator, the report or the archived materials. The agent is trained and evaluated only in simulation. It is not connected to any live system, and it executes no action on the infrastructure of the university.

3.13 Chapter Summary

This chapter described the methodology of the study. The study constructs a simulated triage environment from a scenario model of about 15 causes, 20 diagnostic checks and 10 operational rules elicited from the Directorate of University ICT Services, in which missing, unreliable and overlapping evidence can each be controlled. A reinforcement learning agent chooses among requesting checks, recommending remedies and escalating, and an independent rule-based mechanism reviews each proposal, returns a blocked proposal to the agent once, and escalates the incident if the second proposal is also blocked. The agent is compared with six baseline policies and two ablations on standard measures of decision quality, investigation efficiency, escalation behaviour and constraint compliance, across controlled levels of ambiguity. Chapter Four presents the implementation and the results of these experiments.

Bibliography

[1] Z. Chen *et al.*, "Towards intelligent incident management: Why we need it and how we make it," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2020, pp. 1487–1497. [Online]. Available: https://doi.org/10.1145/3368089.3417055

[2] Y. Remil, A. Bendimerad, R. Mathonat, and M. Kaytoue, "AIOps solutions for incident management: Technical guidelines and a comprehensive literature review," arXiv preprint arXiv:2404.01363, 2024. [Online]. Available: https://arxiv.org/abs/2404.01363

[3] L. Zhang *et al.*, "A survey of AIOps in the era of large language models," *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–35, 2025, art. no. 44. [Online]. Available: https://doi.org/10.1145/3746635

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[5] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998. [Online]. Available: https://doi.org/10.1016/S0004-3702(98)00023-X

[6] J. Janisch, T. Pevný, and V. Lisý, "Classification with costly features using deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, 2019, pp. 3959–3966. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.33013959

[7] L. Wang *et al.*, "Root cause analysis for microservice systems via hierarchical reinforcement learning from human feedback," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD)*, 2023, pp. 5116–5125. [Online]. Available: https://doi.org/10.1145/3580305.3599934

[8] S. Ren, J. Jin, G. Niu, and Y. Liu, "ARCS: Adaptive reinforcement learning framework for automated cybersecurity incident response strategy optimization," *Applied Sciences*, vol. 15, no. 2, p. 951, 2025. [Online]. Available: https://doi.org/10.3390/app15020951

[9] D. Dunsin, M. C. Ghanem, K. Ouazzane, and V. Vassilev, "Reinforcement learning for an efficient and effective malware investigation during cyber incident response," *High-Confidence Computing*, vol. 5, no. 3, p. 100299, 2025. [Online]. Available: https://doi.org/10.1016/j.hcc.2025.100299

[10] Y. Kim and G. Dán, "Dynamic alert prioritization for real-time situational awareness: A hidden Markov model framework with active learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 6, pp. 6913–6928, 2025. [Online]. Available: https://doi.org/10.1109/TDSC.2025.3592127

[11] K. Hammar and T. Li, "Online incident response planning under model misspecification through Bayesian learning and belief quantization," in *Proc. 18th ACM Workshop Artif.*

Intell. Secur. (AISec), 2025, pp. 40–51. [Online]. Available: https://doi.org/10.1145/3733799.3762965

[12] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018. [Online]. Available: https://doi.org/10.1609/aaai.v32i1.11797

[13] B. Könighofer, R. Bloem, N. Jansen, S. Junges, and S. Pranger, "Shields for safe reinforcement learning," *Communications of the ACM*, vol. 68, no. 11, pp. 80–90, 2025. [Online]. Available: https://doi.org/10.1145/3715958

[14] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, no. 42, pp. 1437–1480, 2015.

[15] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 70, 2017, pp. 22–31.

[16] A. Wachi, X. Shen, and Y. Sui, "A survey of constraint formulations in safe reinforcement learning," in *Proc. 33rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024, pp. 8262–8271. [Online]. Available: https://doi.org/10.24963/ijcai.2024/913

[17] W. Zhao, T. He, R. Chen, T. Wei, and C. Liu, "State-wise safe reinforcement learning: A survey," in *Proc. 32nd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2023, pp. 6814–6822. [Online]. Available: https://doi.org/10.24963/ijcai.2023/763

[18] M. Alauthman, A. Almomani, A. Al-Qerem, G. Albesani, and M. Alkasassbeh, "Explainable reinforcement learning for automated incident response in critical infrastructure," in *Proc. 2nd Int. Conf. Comput. Intell. Approaches Appl. (ICCIAA)*, 2026, pp. 1–7. [Online]. Available: https://doi.org/10.1109/ICCIAA68481.2026.11543556

[19] K. Hammar, T. Alpcan, and E. C. Lupu, "Hallucination-resistant security planning with a large language model," in *Proc. IEEE Netw. Oper. Manage. Symp. (NOMS)*, 2026, pp. 1–12. [Online]. Available: https://doi.org/10.1109/NOMS69089.2026.11668410

[20] Y. Geifman and R. El-Yaniv, "SelectiveNet: A deep neural network with an integrated reject option," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 97, 2019, pp. 2151–2159.

[21] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: Improving fairness and accuracy by learning to defer," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[22] P. Hemmer, L. Thede, M. Vössing, J. Jakubik, and N. Kühl, "Learning to defer with limited expert predictions," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 5, 2023, pp. 6002–6011. [Online]. Available: https://doi.org/10.1609/aaai.v37i5.25742

[23] E. Bondi *et al.*, "Role of human-AI interaction in selective prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 5, 2022, pp. 5286–5294. [Online]. Available: https://doi.org/10.1609/aaai.v36i5.20465

[24] S. Joshi, S. Parbhoo, and F. Doshi-Velez, "Learning-to-defer for sequential medical decision-making under uncertainty," *Transactions on Machine Learning Research*, 2023. [Online]. Available: https://openreview.net/forum?id=0pn3KnbH5F

[25] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," National Institute of Standards and Technology,

Gaithersburg, MD, USA, Tech. Rep. NIST AI 100-1, Jan. 2023. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-1

[26] M. Pacella, G. Papadia, and V. Giliberti, "Governed agentic process automation: A floor-safety guarantee for compliance-critical LLM routing," *Algorithms*, vol. 19, no. 8, p. 627, 2026. [Online]. Available: https://doi.org/10.3390/a19080627

[27] O. Bachar *et al.*, "LLM performance predictors: Learning when to escalate in hybrid human-AI moderation systems," arXiv preprint arXiv:2601.07006, 2026. [Online]. Available: https://arxiv.org/abs/2601.07006

[28] E. Turan, "Oversight has a capacity: Calibrating agent guards to a subjective, fatiguing human," arXiv preprint arXiv:2606.08919, 2026. [Online]. Available: https://arxiv.org/abs/2606.08919

[29] F. Şahin and N. S. Mert, "Dual-Threshold Conformal Deferral for Trustworthy Security Alert Triage," *Electronics (Switzerland)*, vol. 15, no. 18, 2026, art. no. 4084. [Online]. Available: https://doi.org/10.3390/electronics15184084

[30] V. Kovtun, A. Huralnyk, L. Husak, and O. Kovaliuk, "Risk-sensitive decision-making in security operations center alert escalation under resource constraints," in *CEUR Workshop Proceedings*, vol. 4242, 2026, pp. 65–83.

[31] C. Opol, "Reinforcement learning for investigating, acting and escalating in IT incident triage and response: A systematic literature review," 2026, unpublished manuscript, Uganda Christian University.

[32] R. D'Antonio and H. Xie, "Human-in-the-loop runbook improvement with agentic support automation," in *Proc. IEEE 7th Int. Conf. Cogn. Mach. Intell. (CogMI)*, 2025, pp. 341–351. [Online]. Available: https://doi.org/10.1109/CogMI67134.2025.00046

[33] J. Wu and M. Gong, "Policy-invisible violations in LLM-based agents," arXiv preprint arXiv:2604.12177, 2026. [Online]. Available: https://arxiv.org/abs/2604.12177

[34] K. Hammar, Y. Li, T. Alpcan, E. C. Lupu, and D. P. Bertsekas, "Adaptive network security policies via belief aggregation and rollout," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 6544–6559, 2026. [Online]. Available: https://doi.org/10.1109/TIFS.2026.3709131

[35] R. Ding *et al.*, "TraceDiag: Adaptive, interpretable, and efficient root cause analysis on large-scale microservice systems," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2023, pp. 1762–1773. [Online]. Available: https://doi.org/10.1145/3611643.3613864

[36] J. Strong, E. Sun, H. Rogers, H. Higham, and J. A. Noble, "Learning to defer: A survey," Preprint, Dec. 2025.

[37] Z. Zhang, C. C. Nguyen, D. Rosewarne, K. Wells, and G. Carneiro, "Fatigue-aware learning to defer via constrained optimisation," arXiv preprint arXiv:2604.00904, 2026. [Online]. Available: https://arxiv.org/abs/2604.00904

[38] E. Okafor, "A leakage-free benchmark of offline policy learning for SOC alert triage," *Machine Learning with Applications*, vol. 25, 2026, art. no. 100984. [Online]. Available: https://doi.org/10.1016/j.mlwa.2026.100984

[39] M. Wen, R. Ehlers, and U. Topcu, "Correct-by-synthesis reinforcement learning with temporal logic constraints," arXiv preprint arXiv:1503.01793, 2015. [Online]. Available: https://arxiv.org/abs/1503.01793

[40] N. Jansen, B. Könighofer, S. Junges, A. C. Serban, and R. Bloem, "Safe reinforcement learning via probabilistic shields," arXiv preprint arXiv:1807.06096, 2019. [Online]. Available: https://arxiv.org/abs/1807.06096

[41] W.-C. Yang, G. Marra, G. Rens, and L. De Raedt, "Safe reinforcement learning via probabilistic logic shields," arXiv preprint arXiv:2303.03226, 2023. [Online]. Available: https://arxiv.org/abs/2303.03226

[42] E. Hamel-De le Court, F. Belardinelli, and A. W. Goodall, "Probabilistic shielding for safe reinforcement learning," arXiv preprint arXiv:2503.07671, 2025. [Online]. Available: https://arxiv.org/abs/2503.07671

[43] E. Hamel-De le Court, T. Badings, A. Abate, F. Belardinelli, and F. Fabiano, "Robust shielding for safe reinforcement learning," arXiv preprint arXiv:2606.00270, 2026. [Online]. Available: https://arxiv.org/abs/2606.00270

[44] S. Pranger and B. Könighofer, "Easy-to-use shielding for reinforcement learning," arXiv preprint arXiv:2606.03804, 2026. [Online]. Available: https://arxiv.org/abs/2606.03804

[45] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2938–2950. [Online]. Available: https://doi.org/10.1145/3744916.3764546

[46] Z. Xiang *et al.*, "GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning," arXiv preprint arXiv:2406.09187, 2025. [Online]. Available: https://arxiv.org/abs/2406.09187

[47] Z. Chen, M. Kang, and B. Li, "ShieldAgent: Shielding agents via verifiable safety policy reasoning," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 267, 2025. [Online]. Available: https://arxiv.org/abs/2503.22738

[48] W. Hu, H. Jing, H. Shi, C. Fan, H. Li, and Y. Song, "OmniCompliance-100K: A multi-domain, rule-grounded, real-world safety compliance dataset," arXiv preprint arXiv:2603.13933, 2026. [Online]. Available: https://arxiv.org/abs/2603.13933

[49] T. Singh, "Autonomous AI Decision Control Tower: A Deterministic Multi-Agent Decision Intelligence Framework for Enterprise NoOps," in *2026 IEEE 1st International Conference on Artificial Intelligence Implementation and Applications, ICAIIA 2026*, 2026, pp. 190–195. [Online]. Available: https://doi.org/10.1109/ICAIIA70172.2026.11691884

[50] R. V. Ravi, K. Priyanga, and M. Adarsh, "SAFE-HGX: A Hierarchical and Safety-Aware Reinforcement Learning Framework for Intrusion Response in Industrial Control Systems," in *2025 1st International Conference on Advancement in Futuristic Technologies, ICAFT 2025*, 2025. [Online]. Available: https://doi.org/10.1109/ICAFT66710.2025.11453104

[51] I. Levy *et al.*, "ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2410.06703

[52] M. Mohammadi, Y. Li, J. Lo, and W. Yip, "Evaluation and benchmarking of LLM agents: A survey," arXiv preprint arXiv:2507.21504, 2025. [Online]. Available: https://arxiv.org/abs/2507.21504

[53] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, 2004. [Online]. Available: https://doi.org/10.2307/25148625

[54] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv preprint arXiv:2407.17032, 2024. [Online]. Available: https://arxiv.org/abs/2407.17032

[55] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. [Online]. Available: https://doi.org/10.1038/nature14236

[56] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[57] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[58] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34, 2021. [Online]. Available: https://arxiv.org/abs/2108.13264